BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Kỹ thuật điện, điện tử

ĐỀ TÀI

Duyệt đồ thị theo chiều sâu bằng đệ quy

Mã đề DT073

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Lê Hoàng Đức Thịnh
Mã số sinh viên	N24DCDT088
Lớp	D24CQDT01-N

THÀNH PHỐ HỒ CHÍ MINH THÁNG 10 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Kỹ thuật điện, điện tử

ĐỀ TÀI

Duyệt đồ thị theo chiều sâu bằng đệ quy

Mã đề DT073

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Lê Hoàng Đức Thịnh
Mã số sinh viên	N24DCDT088
Lớp	D24CQDT01-N
Học phần và nhóm	INT13185 Nhóm 03
Phiên bản báo cáo	Bản nộp cuối tuần 4/4, ngày 30/10/2026

THÀNH PHỐ HỒ CHÍ MINH THÁNG 10 NĂM 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Sinh viên điền theo bảng phân công
Họ tên MSSV lớp	Lê Hoàng Đức Thịnh; N24DCDT088; D24CQDT01-N
Mã đề và tên đề	DT073 — Duyệt đồ thị theo chiều sâu bằng đệ quy
Căn cứ giáo trình	Chương 6, mục 6.3.1, Bài 6.1; trang in 142–143,145; PDF 154–155,157
Ngày giao ngày nộp	Theo thông báo của giảng viên trên Classroom; phiếu không điền ngày cụ thể

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
PHIẾU GIAO ĐỀ TÀI

DT073 INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Sinh viên Lê Hoàng Đức Thịnh MSSV N24DCDT088

Lớp D24CQDT01-N

Duyệt đồ thị theo chiều sâu bằng đệ quy

Chương 6, 6.3.1 Tìm kiếm theo chiều sâu; Bài 6.1; trang in 142–143, 145; trang PDF 154–155, 157

Đề bài

Cài đặt DFS đệ quy để tìm những đỉnh có thể tới từ một nguồn. Sinh viên dùng mảng đánh dấu ngăn việc đi lặp quanh chu trình, ghi thứ tự thăm và quan sát lúc quay về nút gọi trước. Thứ tự láng giềng tăng dần được cố định để kết quả lần vết dễ kiểm tra.

Dữ liệu vào

Nhập đồ thị vô hướng đơn hoặc khai báo các cạnh trong mã; n=1..100, đỉnh 1..n và nguồn s hợp lệ. Sắp danh sách kề tăng dần trước khi duyệt; chỉ xuất các đỉnh đạt được từ s.

Kết quả ra

In thứ tự DFS, mảng đã thăm và các bước đi sâu/quay về trên ví dụ nhỏ. Mỗi đỉnh được xuất đúng một lần, kể cả đồ thị có chu trình.

Ví dụ 1

Dữ liệu vào	Kết quả chính
n=4; canh=[(1,2),(1,3),(2,4)]; s=1	DFS=[1,2,4,3]

Từ 1 chọn 2 trước 3, đi tới 4 rồi quay về để xét nhánh 3 còn lại.

Ví dụ 2

Dữ liệu vào	Kết quả chính
n=3; canh=[(1,2),(2,3),(1,3)]; s=2	DFS=[2,1,3]

Đánh dấu 2 trước khi đi; từ 1 bỏ qua 2 đã thăm và tiếp tục tới 3 theo thứ tự tăng.

DT073 HƯỚNG DẪN THỰC HIỆN

N24DCDT088 Lê Hoàng Đức Thịnh

Thực hiện lần lượt năm bước

1. Đọc giả mã DFS, vẽ đồ thị và ghi thứ tự các láng giềng sẽ xét.

2. Viết danh sách kề và khởi tạo visited=false cho tất cả đỉnh trước một lần chạy. Chạy riêng hàm này trước khi ghép vào chương trình chính.

3. Viết hàm đệ quy: đánh dấu, ghi đỉnh, rồi gọi các láng giềng chưa được thăm. In trạng thái sau mỗi bước trên ví dụ đã cho.

4. Bổ sung thông báo vào/ra hàm ở ví dụ nhỏ để quan sát quá trình quay lui. Nếu kết quả khác dự kiến, xác định bước đầu tiên bị sai rồi sửa.

5. Chạy hai ví dụ cùng năm ca, chỉ rõ đỉnh nào không được thăm nếu nguồn nằm trong thành phần khác. Nộp mã có chú thích các biến và bảng kết quả năm ca kiểm.

Các hàm nên viết

tao_ke(n,canh): dựng đồ thị

dfs!(ke,u,visited,kq): duyệt đệ quy

chay_dfs(ke,s): khởi tạo và trả kết quả

Năm ca cần kiểm tra

1. n=1, s=1 → [1].

2. Đường 1–2–3, s=1 → [1,2,3].

3. Hình sao 1 nối 2,3,4 → [1,2,3,4].

4. n=4, cạnh 1–2, s=3 → [3].

5. Chu trình 1–2–3–1, s=1 → [1,2,3], không lặp.

Phân tích và nhận xét

Đếm số lần thăm đỉnh và số láng giềng được xét. Với danh sách kề, thời gian O(n+m) trên toàn đồ thị; từ một nguồn chỉ xét phần đạt được. Bộ nhớ visited O(n), lời gọi tối đa O(n). Thử n=5,20,50, không đẩy độ sâu quá giới hạn.

Yêu cầu thêm sau khi phần cơ bản chạy đúng

Tùy chọn: in cha của mỗi đỉnh để vẽ cây DFS, cha của nguồn ghi 0.

Sản phẩm: mã nguồn, các kết quả chạy ví dụ và ca kiểm tra, một lần vết bằng tay, báo cáo theo mẫu môn học. Chạy từ bàn phím hoặc truyền trực tiếp các giá trị vào hàm.

Ngày giao ...../...../..... Ngày nộp ...../...../.....

Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên)
Lê Hoàng Đức Thịnh	Hồ Nhựt Minh

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên) .....

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO

Đây là khung trống dùng chung cho mọi đề DT001–DT091. Điền theo phiếu cá nhân; các ô kết quả, ảnh, số liệu, URL và điểm chưa được điền. Không cần thêm bài toán hoặc thuật toán ngoài phiếu.

Bấm vào phần trong ngoặc vuông để thay bằng nội dung của mình. Các ô có thể gõ nhiều dòng; không có khóa chỉnh sửa hay macro. Với khung ảnh, chọn chữ bên trong rồi Chèn → Hình ảnh; đặt ảnh Cùng dòng với văn bản. Khi cần thêm ca/hàm, sao chép dòng tương ứng.

Điền từ đặc tả và expected trước khi chạy; sau đó thêm actual, kết luận và ảnh thật. Xóa câu hướng dẫn/chỗ không áp dụng có giải thích; giữ rubric để giảng viên chấm. Điền cả hai bìa, cập nhật toàn bộ mục lục/danh mục và kiểm PDF cuối trước khi nộp.

Thay mọi phần trong ngoặc vuông bằng nội dung đã thực hiện. Các câu gợi ý chỉ để hướng dẫn, phải xóa hoặc viết lại trước khi nộp. Không giữ các chỗ trống như một kết quả hoàn thành.

Giữ cấu trúc theo mẫu môn: hai bìa, phiếu, rubric, cam đoan, cảm ơn nếu dùng, tóm tắt, mục lục, danh mục, năm chương, tham khảo và phụ lục. Đây là khung theo hình thức đồ án dùng cho học phần.

Nội dung thân bài dùng style Nội dung: A4; lề trái 3 cm, các lề khác 2 cm; Times New Roman 13; giãn dòng 1,5. Chương đầu trang, hình có chú thích dưới, bảng có tên trên; số theo chương.

Ảnh và bảng phải được nhắc và giải thích trong nội dung. Chèn ảnh In Line with Text, đọc được trên bản in, không kéo méo. Hình lý thuyết không thay ảnh thực thi.

Mở 00_HUONG_DAN_SINH_VIEN.html và rubric chung trước khi viết; tra mã đề để chọn bất biến, kết quả và ca biên phù hợp. Không thêm thuật toán thứ hai nếu phiếu không yêu cầu.

LỜI CAM ĐOAN

Báo cáo được xây dựng theo phiếu DT073 và mẫu của giảng viên; có sử dụng ChatGPT hỗ trợ xây dựng mã, kiểm thử, giải thích và biên tập. Ảnh và log máy cá nhân là căn cứ của phần kết quả thực thi. Tôi có trách nhiệm kiểm tra, hiểu và giải thích thuật toán khi bảo vệ; khai báo trung thực nguồn tham khảo và công cụ hỗ trợ.

TP. Hồ Chí Minh ngày ..... tháng ..... năm .....
Sinh viên thực hiện
(Ký và ghi rõ họ tên)

Lê Hoàng Đức Thịnh

LỜI CẢM ƠN

Em cảm ơn thầy Hồ Nhựt Minh đã cung cấp đề tài, giáo trình và hướng dẫn kiểm chứng sản phẩm. Việc đối chiếu thứ tự duyệt, trạng thái visited và bản clone giúp em nhận ra sự khác nhau giữa chạy đúng trên máy cá nhân và chuẩn bị đầy đủ một bộ mã có thể chạy lại.

TÓM TẮT

Đề tài cài đặt DFS đệ quy trên đồ thị vô hướng đơn có tối đa 100 đỉnh. Danh sách kề được sắp tăng, mảng visited ngăn thăm lặp và mảng cha hỗ trợ dựng cây DFS. Chương trình chỉ xuất thành phần đạt được từ nguồn đã chọn. Bộ kiểm thử gồm 22 ca độc lập, bao gồm hai ví dụ, năm ca trên phiếu, ca tự thiết kế, ca biên và dữ liệu sai. Khảo sát sử dụng bộ đếm tại ba quy mô hợp lệ.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI.....i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG.....iv
CÁCH ĐỐI CHIẾU RUBRIC.....v
HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO.....vi
LỜI CAM ĐOAN.....vii
LỜI CẢM ƠN.....viii
TÓM TẮT.....ix
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....1
1.1. Bài toán và lý do thực hiện.....1
1.2. Mục tiêu và sản phẩm.....1
1.3. Phạm vi, đầu vào và đầu ra.....1
1.4. Cách thực hiện và cấu trúc báo cáo.....2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....3
2.1. Khái niệm và mô hình dữ liệu.....3
2.2. Cấu trúc dữ liệu sử dụng.....3
2.3. Nguyên lý thuật toán.....5
2.4. Cơ sở đánh giá thời gian và bộ nhớ.....5
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT.....6
3.1. Phân tích yêu cầu chương trình.....6
3.2. Thiết kế các hàm và dữ liệu.....6
3.3. Thuật toán và mã nguồn chính.....7
3.4. Lần vết ví dụ nhỏ.....9
3.5. Giải thích tính đúng và điều kiện dừng.....10
3.6. Phân tích độ phức tạp.....10
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT.....12
4.1. Môi trường và kế hoạch kiểm thử.....12
4.2. Bảng kết quả ít nhất 12 ca.....12
4.3. Hai ví dụ trên phiếu.....16
4.4. Hai ca sinh viên tự thiết kế.....18
4.5. Ba ca biên phù hợp với đề tài.....20
4.6. Kiểm tra input không hợp lệ — LOI01.....22
4.7. Kết quả toàn bộ kiểm thử.....23
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ.....24
4.9. GitHub và chạy lại từ bản clone sạch.....26
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....29
5.1. Mức hoàn thành so với phiếu.....29
5.2. Giới hạn và bài học.....29
5.3. Hướng phát triển phù hợp.....29
TÀI LIỆU THAM KHẢO.....30
PHỤ LỤC.....PL-1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy.....PL-1
PHỤ LỤC B. Danh mục bằng chứng.....PL-2
PHỤ LỤC C. Tự kiểm trước khi nộp.....PL-3
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM.....PL-4

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
n; m	Số đỉnh và số cạnh
ke; visited	Danh sách kề; mảng đã thăm
s; cha	Nguồn; mảng cha
C; S	Commit mã chạy và commit nộp cuối

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào....1
Bảng 1.2 Đặc tả kết quả đầu ra....2
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm....2
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì....3
Bảng 3.1 Tổ chức các hàm và tệp....6
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm....7
Bảng 3.3 Truy vết ca VD01....9
Bảng 3.4 Chi phí của từng pha trong bản cài....10
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu....12
Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập....13
Bảng 4.3 Actual kết luận và vị trí minh chứng....15
Bảng 4.4 Khảo sát Q1 Q2 Q3....25
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại....26
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu....29
Bảng PL.1 Bốn tệp cần nộp....PL-4

DANH SÁCH HÌNH

Hình 2.1 Đồ thị VD01 và thứ tự DFS....4
Hình 4.1 Ví dụ 1 VD01....16
Hình 4.2 Ví dụ 2 VD02....17
Hình 4.3 Ca tự thiết kế 1 TK01....18
Hình 4.4 Ca tự thiết kế 2 TK02....19
Hình 4.5 Biên 1 BIEN01....20
Hình 4.6 Biên 2 BIEN02....21
Hình 4.7 Biên 3 BIEN03....22
Hình 4.8 Input không hợp lệ LOI01....23
Hình 4.9 Tổng kết kiểm thử TEST_ALL....24
Hình 4.10 Clone sạch và chạy lại CLONE....27
Hình 4.11 GitHub repo và commit GITHUB....28

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Nêu bài toán đúng tên đề và bối cảnh nhỏ phù hợp; chỉ ra kết quả cần tính. Trích nguồn giáo trình tại đoạn dùng. Không cần tạo một hệ thống ứng dụng lớn ngoài yêu cầu.

Duyệt đồ thị giúp xác định các đỉnh liên hệ với một nguồn. Với đồ thị có chu trình, việc đánh dấu đúng thời điểm là điều kiện để DFS dừng và không xuất đỉnh lặp. Đề tài tập trung vào cách tổ chức lời gọi đệ quy và quá trình trở về nhánh trước.

1.2. Mục tiêu và sản phẩm

Liệt kê từng chức năng bắt buộc từ phiếu; với mỗi chức năng, nêu output hoặc điều kiện hoàn thành kiểm được. Nêu mã chạy, bộ test, báo cáo và repo.

Sản phẩm gồm mã thuật toán, chương trình nhập/chạy ví dụ, bộ kiểm thử độc lập, lần vết, khảo sát ba quy mô và báo cáo. Phần mở rộng chỉ bổ sung sau khi chức năng cơ bản đã kiểm đúng: ghi cha của mỗi đỉnh để dựng cây DFS.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Ghi kiểu dữ liệu, miền giá trị, chỉ số và thứ tự nhập; nêu rõ nhập bàn phím/khai báo trực tiếp.

Đồ thị vô hướng đơn, n nguyên thuộc 1..100, các đỉnh 1..n, nguồn s hợp lệ. Cạnh có hai đầu mút nguyên khác nhau và không trùng kể cả đảo chiều. Có thể nhập bàn phím, JSON hoặc khai báo cạnh trong mã.

1.3.2. Kết quả và trường hợp đặc biệt

Ghi dạng output, quy tắc hòa nếu có, trường hợp không có nghiệm và cách xử lý input sai.

Trả thứ tự DFS, visited, cha, bộ đếm và trace tùy chọn. Nguồn cô lập cho kết quả [s]; đỉnh ở thành phần khác giữ False. Thứ tự láng giềng tăng cố định, không có hòa; input sai báo ValueError.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
n	int	1..100	Bàn phím/JSON
canh	Danh sách cặp	Vô hướng đơn, đỉnh 1..n	Bàn phím/JSON
s	int	1..n	Bàn phím/JSON

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
thu_tu; visited	list[int]; list[bool]	Mỗi đỉnh đạt được xuất đúng một lần	Nguồn cô lập → [s]
cha; dem; vet	list[int]; dict; list[str]	Cha nguồn/chưa tới=0; bộ đếm và trace	Dùng visited phân biệt; input sai ValueError

Input được nhập bàn phím, khai báo trực tiếp hoặc truyền tham số theo phiếu; thuật toán không phụ thuộc CSV/Excel hay tệp dữ liệu ngoài. Nếu trường hợp hòa/vô nghiệm/rỗng không áp dụng, ghi Không áp dụng và nêu lý do.

1.4. Cách thực hiện và cấu trúc báo cáo

Tóm tắt đọc sách → làm tay → cài đặt → kiểm thử → bằng chứng; giới thiệu mối liên hệ năm chương.

Từ phiếu giao đề, xác định hợp đồng và kỳ vọng trước khi viết test. Cài thuật toán, đối chiếu lần vết, chạy kiểm thử và khảo sát bằng bộ đếm. Chương 2 trình bày cơ sở; Chương 3 thiết kế và chứng minh; Chương 4 kết quả thực thi; Chương 5 kết luận và giới hạn.

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Thuật toán và kết quả đúng phiếu	Hàm chính trong src/dfs.py	Chương 3
Hai ví dụ và năm ca bắt buộc	Ca VD/KT/BIEN và bảng đối chiếu	4.2–4.7
Lần vết, tính đúng, khảo sát	Bảng truy vết, lập luận và Q1–Q3	3.4–3.6; 4.8

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Định nghĩa các đối tượng của đúng đề: số, mảng, dãy con, trạng thái, nút, cây hoặc đồ thị. Ghi ký hiệu n và tham số phụ; dẫn mục/trang giáo trình.

Theo giáo trình, DFS đi sâu tới láng giềng chưa thăm, sau đó trở về để xét nhánh còn lại [1, mục 6.3.1, trang in 142–143]. Đồ thị vô hướng có mỗi cạnh xuất hiện trong danh sách kề của hai đầu mút. Mảng đánh dấu được giữ xuyên suốt một lần duyệt; không bỏ đánh dấu khi quay về.

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Nêu mỗi cấu trúc lưu gì, thao tác nào dùng và lý do phù hợp. Đặt sơ đồ nhỏ nếu làm rõ được liên hệ.

Dùng list gồm n+1 danh sách kề, visited và cha. Set kiểm cạnh trùng; sort chuẩn hóa thứ tự. DFS đệ quy tự cài; không gọi thư viện duyệt đồ thị.

2.2.2. Quy ước và điều kiện hợp lệ

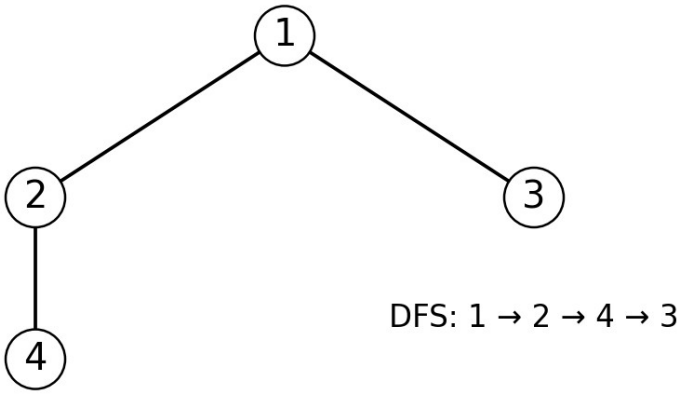
Nêu chỉ số, ký hiệu rỗng, khóa trùng, hướng cạnh hoặc điều kiện tương ứng đúng phiếu. Bỏ những khái niệm không dùng.

Ô 0 của ke bỏ trống, ke[u] tăng và đối xứng với cạnh vô hướng. visited được đánh dấu trước khi đệ quy và không xóa khi quay về. Mỗi lần chay_dfs khởi tạo visited mới.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
ke	Mảng các danh sách kề	1..n; tăng dần, đối xứng
visited	Đỉnh đã thăm	Đánh dấu trước khi đệ quy
kq	Thứ tự thăm	Mỗi đỉnh xuất một lần
cha	Đỉnh gọi DFS tới v	Nguồn/chưa tới có cha=0
dem	Đỉnh, láng giềng, độ sâu	Đếm tại mỗi sự kiện thực tế

Sơ đồ hoặc ví dụ mô hình



Hình 2.1 Đồ thị VD01 và thứ tự DFS.

Cơ sở thuật toán và giới hạn áp dụng

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Mô tả bước lựa chọn/cập nhật cốt lõi của đề; với nhiều phương án theo phiếu, tách từng phương án và điểm khác. Nếu một phương án thì chỉ trình bày phương án đó.

Khởi tạo visited=False và kq rỗng cho mỗi lần chạy. DFS(u) đánh dấu u, ghi u, xét lần lượt ke[u] tăng dần và đệ quy vào v chưa thăm. Khi hết láng giềng, hàm trở về nơi gọi; visited vẫn giữ nguyên. Không tự chuyển sang thành phần liên thông khác.

2.3.2. Điều kiện bắt đầu và kết thúc

Nêu dữ liệu khởi tạo, điều kiện dừng và kết quả trả về. Với đệ quy/quy hoạch động, nêu cơ sở và quan hệ truy hồi; với cấu trúc, nêu bất biến thao tác.

Khởi tạo visited=False, kq rỗng; gọi dfs tại s. Mỗi lời gọi đánh dấu thêm một đỉnh mới; tối đa n lời gọi. Hết danh sách kề thì trở về, không bỏ đánh dấu và không chuyển sang thành phần khác.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Định nghĩa quy mô và phép toán chính; nêu dự đoán O theo giáo trình cho các phần sẽ cài. Phân biệt bộ nhớ input và bộ nhớ phụ, chi phí lõi và in kết quả. Mọi khẳng định có điều kiện áp dụng cụ thể.

Gọi n_r,m_r là số đỉnh và cạnh của thành phần chứa s. Sau khi có danh sách kề đã sắp, lõi duyệt xét n_r đỉnh và $2m_r$ mục kề. Khởi tạo mảng vẫn tốn O(n). Chi phí dựng, kiểm cạnh và sắp xếp cần phân tích riêng, không gộp tất cả thành O(n+m) mà bỏ qua sort.

Giả mã nền tảng

Giả mã theo giáo trình [1] và trạng thái của phiếu [2]. Đánh dấu, ghi đỉnh, rồi đệ quy vào các láng giềng chưa thăm theo thứ tự tăng.

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Viết hợp đồng hàm bằng tên biến thật, điều kiện đầu vào, giá trị trả và lỗi. Đối chiếu từng output với phiếu.

tao_ke(n,canh) kiểm n và từng cạnh, dựng danh sách kề vô hướng rồi sắp tăng. chay_dfs(ke,s, *, lan_vet=False) nhận ke hợp lệ do tao_ke tạo, khởi tạo mới các mảng và trả dict {thu_tu,visited,cha,dem,vet}. dfs(ke,u,visited,kq,cha,dem,vet=None,sau=1) cập nhật trạng thái theo tham chiếu và trả None; không phải hàm xác thực input độc lập.

3.1.2. Kiểm tra đầu vào

Chỉ rõ điều kiện cần xác thực và hành vi khi vi phạm. Phân biệt bài toán hợp lệ không có nghiệm với input sai.

Kiểm tra đầu vào tại hàm công khai trước khi đệ quy. Chỉ chấp nhận type(x) is int để không nhận True/False như số nguyên. CLI hiển thị thông báo input không hợp lệ và mã thoát 2; runner kiểm thử trả 1 nếu có ca FAIL. Một ca cố ý input sai đạt PASS khi chương trình từ chối đúng.

3.2. Thiết kế các hàm và dữ liệu

Nêu tên hàm, tham số, giá trị trả về và trách nhiệm. Chỉ ra phần bắt buộc tự cài và công cụ hỗ trợ có sẵn. Tên hàm trong bảng phải có trong mã thật.

Các tệp src/, demo.py, test/run_tests.py và survey.py lần lượt đảm nhiệm thuật toán, giao tiếp, kiểm thử và khảo sát. Tên hàm, tham số và kết quả trong các bảng dưới đối chiếu với mã Python kèm theo. Thư viện chuẩn chỉ hỗ trợ nhập/xuất, tổ chức dữ liệu và kiểm thử; thuật toán chính được cài bằng các bước của phiếu.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
tao_ke(n,canh)	n và các cạnh	ke đã sắp tăng	Kiểm input, dựng kề vô hướng
chay_dfs(ke,s,…)	ke chuẩn, s hợp lệ	Dict thứ tự, visited, cha, đếm, trace	Khởi tạo mới mỗi lượt
dfs(ke,u,…)	u chưa thăm	Cập nhật visited,kq,cha	Đệ quy đi sâu và trở về

Sơ đồ xử lý

Nhập n,canh,s → tao_ke và kiểm tra → sắp kề → chay_dfs → dfs → in thứ tự, visited và cha.

Cài đặt phần quyết định

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
tao_ke(n,canh)	n nguyên 1..100; cạnh là cặp nhãn hợp lệ, không khuyên/trùng	ke có n+1 ô, bỏ ô 0; hai chiều, sắp tăng; không sửa canh	ValueError khi n/cạnh sai hợp đồng
chay_dfs(ke,s, *, lan_vet=False)	ke do tao_ke tạo; s nguyên 1..n	Trả thứ tự, visited, cha, đếm, trace; mỗi lần khởi tạo mới	ValueError nếu s sai; nguồn cô lập trả [s]
dfs(ke,u,visited,kq,cha,dem,vet,sau)	ke hợp lệ; u chưa thăm; các mảng đã khởi tạo	Đánh dấu, ghi u; duyệt sâu; giữ visited khi trở về	Hàm nội bộ; không tự kiểm mọi input; trả None

Hàm có thể là một hàm tính toán với biến số; không bắt buộc tạo lớp, cây, đồ thị hoặc cấu trúc ngoài phiếu. Ghi rõ dữ liệu nào bị thay đổi và thao tác nào phải tự cài.

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

Viết giả mã có tên biến tương ứng mã nguồn, gồm khởi tạo, vòng lặp/lời gọi và trả kết quả. Nếu nhiều cách theo phiếu, bổ sung các tiểu mục riêng.

dfs(u): visited[u]=True; ghi u; với v trong ke[u] tăng dần, nếu chưa thăm thì cha[v]=u và gọi dfs(v). Hết láng giềng thì trở về.

3.3.2. Đoạn mã và giải thích

Trích đoạn mã quyết định; giải thích biến, thứ tự cập nhật và lỗi có thể xảy ra. Ghi permalink tệp tại commit chạy C; toàn bộ mã nằm trong repo.

visited[u]=True được thực hiện trước khi xét láng giềng để ngăn việc gọi lặp quanh chu trình. kq.append(u) ghi thứ tự khám phá; sort trong tao_ke cố định thứ tự xét. cha[v]=u chỉ được gán khi v chưa thăm. Khi trở về, không xóa visited vì đây là tập đỉnh đã khám phá của toàn lần duyệt.
Permalink mã tại C:
https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh/blob/0347b0a26d4137b2efe6a4a37920262f9d85091b/src/dfs.py

3.3.3. Xử lý tình huống đặc biệt

Nêu nhánh xử lý các biên của chính đề và input sai; dẫn mã ca sẽ kiểm ở Chương 4.

Một đỉnh, nguồn cô lập và đường đi 100 đỉnh được xử lý; chu trình không lặp. Khuyên, cạnh trùng, cạnh sai và nguồn ngoài miền bị từ chối; xem BIEN01–03, LOI01, KT13–17, KT21–22.

Giả mã

```
DFS(u):
  Đánh dấu visited[u]=True; ghi u vào kq.
  Tăng số đỉnh thăm; cập nhật độ sâu.
  Với v trong ke[u] đã sắp tăng:
    Tăng số mục kề đã xét.
    Nếu visited[v]=False:
      Gán cha[v]=u; gọi DFS(v).
  Hết danh sách kề: trở về nơi gọi.
Mỗi lần chạy: khởi tạo visited=False, cha=0, kq=[]; gọi DFS(s).
```

Đoạn mã có phân tích

```
visited[u] = True
kq.append(u)
dem['so_dinh_tham'] += 1
dem['do_sau'] = max(dem['do_sau'], sau)
for v in ke[u]:
    dem['so_lang_gien_xet'] += 1
    if not visited[v]:
        cha[v] = u
        dfs(ke, v, visited, kq, cha, dem, vet, sau+1)
```

Trích lõi src/dfs.py; các thông báo vào/đi/bỏ/ra được ghi khi bật lan_vet. Permalink: https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh/blob/0347b0a26d4137b2efe6a4a37920262f9d85091b/src/dfs.py

3.4. Lần vết ví dụ nhỏ

Ghi mã ca, toàn bộ input và expected. Điền bảng từng bước từ khởi tạo đến kết thúc; giải thích bước đổi trạng thái quan trọng nhất. Dẫn ảnh chạy đúng cùng input ở Chương 4.

VD01: n=4, E={(1,2),(1,3),(2,4)}, s=1. Danh sách kề là 1:[2,3], 2:[1,4], 3:[1], 4:[2]. Thứ tự dự kiến [1,2,4,3]; visited cuối là [True,True,True,True], cha=[0,1,1,2]. Trong bảng, V là tập đỉnh đã thăm, K là dãy kết quả, S là ngăn xếp các lời gọi DFS đang hoạt động; các chữ số được viết liền cho gọn.

Ca truy vết VD01. Input: n=4; E=[[1, 2], [1, 3], [2, 4]]; s=1. Expected: [1, 2, 4, 3]

Bảng 3.3 Truy vết ca VD01

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
0	Khởi tạo	s=1	V=∅; K=∅; S=∅	V: đã thăm; K: thứ tự; S: khung DFS
1	Vào 1	V=∅; K=∅; S=∅	V=1; K=1; S=1	Đánh dấu rồi ghi đỉnh
2	Đi 1→2	V=1; K=1; S=1	V=1; K=1; S=1	Gán cha[2]=1; chuẩn bị gọi
3	Vào 2	V=1; K=1; S=1	V=12; K=12; S=12	Đánh dấu rồi ghi đỉnh
4	Bỏ cạnh 2→1	V=12; K=12; S=12	V=12; K=12; S=12	Đỉnh cuối đã thăm
5	Đi 2→4	V=12; K=12; S=12	V=12; K=12; S=12	Gán cha[4]=2; chuẩn bị gọi
6	Vào 4	V=12; K=12; S=12	V=124; K=124; S=124	Đánh dấu rồi ghi đỉnh
7	Bỏ cạnh 4→2	V=124; K=124; S=124	V=124; K=124; S=124	Đỉnh cuối đã thăm
8	Ra 4	V=124; K=124; S=124	V=124; K=124; S=12	Giữ V và K; trả về nơi gọi
9	Ra 2	V=124; K=124; S=12	V=124; K=124; S=1	Giữ V và K; trả về nơi gọi
10	Đi 1→3	V=124; K=124; S=1	V=124; K=124; S=1	Gán cha[3]=1; chuẩn bị gọi
11	Vào 3	V=124; K=124; S=1	V=1234; K=1243; S=13	Đánh dấu rồi ghi đỉnh
12	Bỏ cạnh 3→1	V=1234; K=1243; S=13	V=1234; K=1243; S=13	Đỉnh cuối đã thăm

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
13	Ra 3	V=1234; K=1243; S=13	V=1234; K=1243; S=1	Giữ V và K; trả về nơi gọi
14	Ra 1	V=1234; K=1243; S=1	V=1234; K=1243; S=∅	Giữ V và K; trả về nơi gọi

Từ 1 đi 2 trước, từ 2 bỏ 1 đã thăm và đi 4; quay về 2 rồi 1 trước khi tới 3. visited không bị xóa. Kết quả [1,2,4,3] khác thứ tự BFS [1,2,3,4].

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Nêu bất biến hoặc quan hệ truy hồi; giải thích vì sao một bước giữ tính chất đúng.

Bất biến: kq chứa đúng các đỉnh đã đánh dấu theo thứ tự khám phá; mỗi đỉnh được ghi một lần. Đầu lời gọi, đánh dấu u rồi mới xét láng giềng. DFS(v) chỉ được gọi khi v chưa thăm, nên chu trình không làm lặp đỉnh. Nguồn đạt được chính nó; mỗi đỉnh mới được gọi qua một cạnh từ đỉnh đã tới, do đó mọi đỉnh ghi ra đều đạt được từ s.

3.5.2. Kết thúc và output

Chỉ ra đại lượng làm thuật toán tiến tới dừng; suy ra kết quả trả đúng yêu cầu. Kiểm thử bổ sung bằng chứng, không thay lập luận.

Mỗi lời gọi đánh dấu một đỉnh mới, nên có tối đa n lời gọi DFS và mỗi danh sách kề hữu hạn; thuật toán dừng. Để chứng minh đủ, giả sử còn đỉnh đạt được từ s nhưng chưa thăm. Trên đường từ s tới đỉnh ấy phải có cạnh đầu tiên nối đỉnh đã thăm với đỉnh chưa thăm. DFS của đỉnh đầu phải xét cạnh này và gọi đỉnh sau trước khi trở về, mâu thuẫn. Vì vậy kết quả gồm đúng các đỉnh thuộc thành phần nguồn, không lặp.

3.6. Phân tích độ phức tạp

Phân tích theo phép toán chính đã định nghĩa, tách các pha nếu cần; nêu bộ nhớ phụ. Dự đoán xu hướng cho ba quy mô hoặc cấu hình hợp lệ sẽ khảo sát ở Chương 4.

Đặt n_r,m_r là số đỉnh/cạnh trong thành phần nguồn. Lõi DFS tốn $O(n_r+m_r)$, xét đúng $2m_r$ mục kề; chay_dfs còn khởi tạo mảng dài n nên tốn $O(n+n_r+m_r)$. Bộ nhớ phụ visited, cha, kết quả và ngăn xếp là O(n), ngoài danh sách kề O(n+m). tao_ke có chi phí sắp riêng $\Sigma d_u \log(\max(2,d_u))$; phép kiểm trùng bằng set có chi phí kỳ vọng O(m). Các chặn trong bảng áp dụng khi tắt trace và phân tích set theo kỳ vọng.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Kiểm tra và chuẩn bị	n,m,d_u	Cạnh, dựng kề và sort	$O(n+m+\Sigma d_u \log(\max(2,d_u)))$	O(n+m)

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Thuật toán chính	n,n_r,m_r	Đỉnh thăm và mục kề xét	$O(n+n_r+m_r)$	O(n) phụ
Kiểm nghiệm hoặc truy vết nếu có	Số sự kiện	Lưu trạng thái trace	Theo độ dài chuỗi log	Theo log lưu
In và định dạng kết quả	n,n_r	visited, cha, thứ tự	$O(n+n_r)$	Bộ đệm dòng
Toàn chương trình	n,m	Tắt trace; gồm sort	$O(n+m+\Sigma d_u \log(max(2,d_u)))$	O(n+m) gồm ke

Tách lõi thuật toán khỏi chuẩn bị/sắp/kiểm/in khi các bước đó tồn tại. Nêu bộ nhớ phụ và cách tính kích thước kết quả; không khẳng định chi phí chỉ từ số đo nhỏ. Dòng không áp dụng ghi rõ lý do.

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Ghi môi trường, phiên bản, đường dẫn chương trình/test và SHA chạy C. SHA nộp cuối S ghi trong ThongTinNop.txt nộp rời.

Môi trường thực chạy: Windows 11; Python 3.14.5; Git 2.55.0.windows.3.
Log cá nhân: Windows-11-10.0.26300-SP0.
URL repo:
https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh
SHA C theo ảnh A10a: 0347b0a26d4137b2efe6a4a37920262f9d85091b

4.1.2. Phương pháp xác định kỳ vọng

Nêu tính tay, cách giải độc lập hoặc điều kiện nghiệm; giải thích cách so sánh khi nhiều nghiệm và sai số nếu dùng số thực.

Thứ tự expected được tính tay theo kề tăng, không lấy chính output DFS làm đáp án. Để đối chứng tập đỉnh đạt được, bắt đầu với R={s}, quét các cạnh và thêm hai đầu khi một đầu đã thuộc R; lặp đến khi R không đổi. Cách này độc lập với đệ quy DFS. So sánh thêm visited, không trùng, cha hợp lệ và số mục kề xét; đối chứng tập đạt được không thay kiểm thứ tự DFS.

4.2. Bảng kết quả ít nhất 12 ca

Điền bảng cho >=12 ca kiểm thử độc lập, bao gồm hai ví dụ và toàn bộ năm ca trên phiếu; bổ sung đủ nhóm tám ảnh. Có thể xoay bảng ở trang riêng nếu cần, giữ cỡ chữ đọc được. Không dùng số assert thay số ca.

Bộ kiểm thử gồm 22 ca độc lập, kết quả máy cá nhân là 22 PASS, 0 FAIL và 234 phép kiểm. Bảng 4.2–4.3 giữ 12 dòng theo mẫu; KT13–KT22 được lưu đầy đủ trong log máy cá nhân. Các ca trên phiếu đều có ánh xạ ở Bảng 4.1.

Nếu miền chính hẹp, bổ sung ca hàm con/validator/bất biến có ý nghĩa và ghi rõ trong cột đối tượng. Không ép 12 input chính hợp lệ khác nhau, không bỏ các ca trên phiếu, không tính lặp cùng phép kiểm.

Ca helper/validator có mục tiêu riêng, không tính lại cùng input và cùng phép kiểm để tăng số ca.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Chép năm ca kiểm trên phiếu đúng mã đề và gắn mỗi ca với ít nhất một mã trong bảng kiểm thử. Hai ví dụ trên phiếu cũng phải có mặt. Một ca có thể đáp ứng nhiều yêu cầu nhưng cùng input, mục tiêu và phép kiểm chỉ đếm một lần. Không bỏ ca bắt buộc để giữ đúng 12 dòng: 12 là số ca độc lập tối thiểu; thêm dòng khi cần.

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 1	n=1,s=1 → [1]	BIEN01	Log máy cá nhân: BIEN01 PASS; ảnh A09a/A09b.
Ca phiếu 2	Đường 1–2–3,s=1 → [1,2,3]	KT09	Log máy cá nhân: KT09 PASS; ảnh A09a/A09b.
Ca phiếu 3	Sao 1 nối 2,3,4 → [1,2,3,4]	KT10	Log máy cá nhân: KT10 PASS; ảnh A09a/A09b.
Ca phiếu 4	n=4,E={(1,2)},s=3 → [3]	BIEN03	Log máy cá nhân: BIEN03 PASS; ảnh A09a/A09b.
Ca phiếu 5	Tam giác,s=1 → [1,2,3], không lặp	KT11	Log máy cá nhân: KT11 PASS; ảnh A09a/A09b.

Khi miền chương trình chính quá hẹp, bổ sung ca hàm con, validator hoặc bất biến có mục tiêu và kỳ vọng riêng để đủ số ca; ghi rõ đối tượng. Không đổi nhãn cùng phép kiểm để tăng số ca và không tính số assert là số ca.

Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
VD01	Ví dụ 1: nhánh sâu trước nhánh 3	n=4; E=[[1, 2], [1, 3], [2, 4]]; s=1	[1, 2, 4, 3]
VD02	Ví dụ 2: tam giác từ nguồn 2	n=3; E=[[1, 2], [2, 3], [1, 3]]; s=2	[2, 1, 3]
TK01	Hai thành phần, chỉ duyệt phần chứa nguồn	n=6; E=[[1, 2], [2, 3], [4, 5]]; s=4	[4, 5]
TK02	Cạnh nhập đảo thứ tự vẫn sắp láng giềng	n=5; E=[[1, 5], [3, 4], [1, 3], [1, 2]]; s=1	[1, 2, 3, 4, 5]
BIEN01	Ca phiếu 1: một đỉnh	n=1; E=[]; s=1	[1]
BIEN02	Biên n=100 và độ sâu lớn nhất	n=100; E={(i,i+1): i=1..99}; s=1	[1,2,…,100]
BIEN03	Ca phiếu 4: nguồn cô lập	n=4; E=[[1, 2]]; s=3	[3]
LOI01	Nguồn không hợp lệ	n=3; E=[[1, 2]]; s=0	Từ chối bằng ValueError
KT09	Ca phiếu 2: đường đi	n=3; E=[[1, 2], [2, 3]]; s=1	[1, 2, 3]
KT10	Ca phiếu 3: hình sao	n=4; E=[[1, 2], [1, 3], [1, 4]]; s=1	[1, 2, 3, 4]

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
KT11	Ca phiếu 5: chu trình không lặp	n=3; E=[[1, 2], [2, 3], [3, 1]]; s=1	[1, 2, 3]
KT12	Chạy lại từ nguồn khác phải reset visited	n=4; E=[[1, 2], [3, 4]]; s=3	[3, 4]

Bảng 4.3 Actual kết luận và vị trí minh chứng

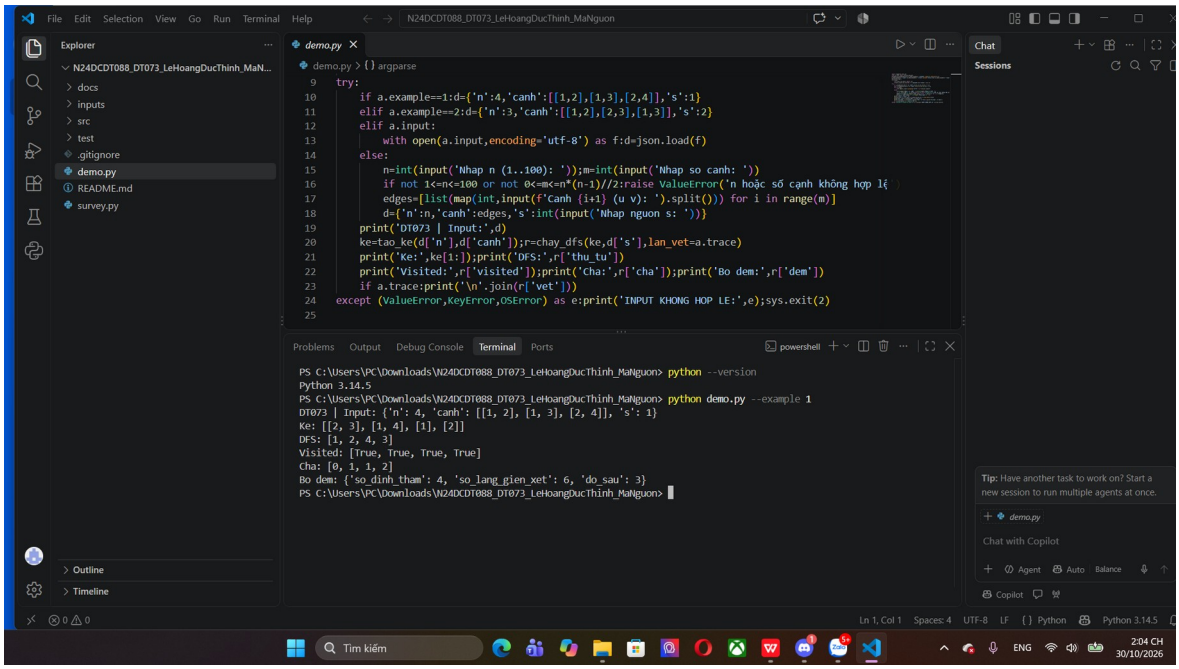
Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01	Thứ tự: [1, 2, 4, 3]	PASS	A01 / kiểm lại tại C: 0347b0a
VD02	Thứ tự: [2, 1, 3]	PASS	A02 / kiểm lại tại C: 0347b0a
TK01	Thứ tự: [4, 5]	PASS	A03 / kiểm lại tại C: 0347b0a
TK02	Thứ tự: [1, 2, 3, 4, 5]	PASS	A04 / kiểm lại tại C: 0347b0a
BIEN01	Thứ tự: [1]	PASS	A05 / kiểm lại tại C: 0347b0a
BIEN02	Thứ tự [1,2,…,100]; 100 đỉnh được thăm.	PASS	A06 / kiểm lại tại C: 0347b0a
BIEN03	Thứ tự: [3]	PASS	A07 / kiểm lại tại C: 0347b0a
LOI01	ValueError: nguồn s phải thuộc 1..n	PASS	A08 / kiểm lại tại C: 0347b0a
KT09	Thứ tự: [1, 2, 3]	PASS	Log máy cá nhân; kiểm lại tại C: 0347b0a
KT10	Thứ tự: [1, 2, 3, 4]	PASS	Log máy cá nhân; kiểm lại tại C: 0347b0a
KT11	Thứ tự: [1, 2, 3]	PASS	Log máy cá nhân; kiểm lại tại C: 0347b0a
KT12	Thứ tự: [3, 4]	PASS	Log máy cá nhân; kiểm lại tại C: 0347b0a

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Input: n=4; E={(1,2),(1,3),(2,4)}; s=1. Expected: [1,2,4,3], xác định bằng lần vết kề tăng. Lệnh chạy: python demo.py --example 1. Actual: [1,2,4,3]. Chèn A01 và viết >=2 câu nhận xét, dẫn bảng lần vết nếu dùng cùng input.

VD01. Input: n=4; E=[[1, 2], [1, 3], [2, 4]]; s=1. Expected: [1, 2, 4, 3].
Lệnh: python demo.py --example 1
Actual: Thứ tự: [1, 2, 4, 3]
Kết quả được kiểm lại tại C trong A10a/A10b.



Hình 4.1 Ví dụ 1 VD01

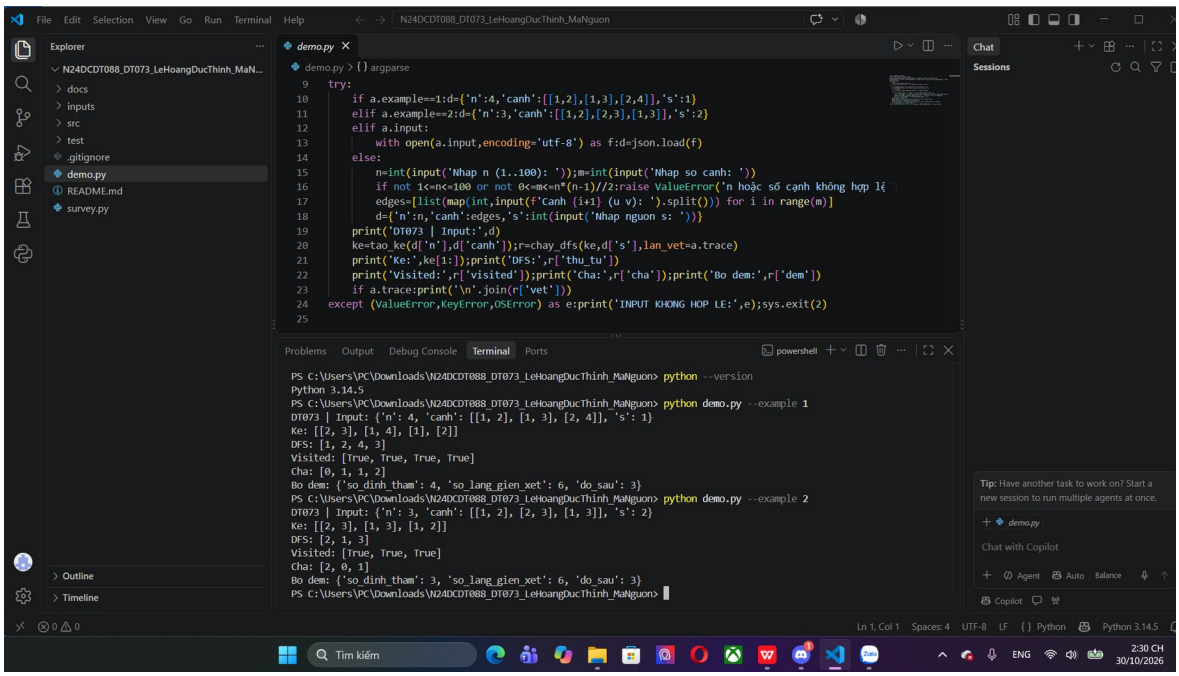
Ảnh: docs/anh_ket_qua/A01.png. Kề của đỉnh 1 là [2,3], vì vậy DFS đi $1 \to 2 \to 4$ rồi trở về mới thăm 3. Kết quả [1,2,4,3] cùng cha=[0,1,1,2] kiểm được cả đi sâu và quay về.

4.3.2. Ví dụ 2 — VD02

Input: n=3; E={(1,2),(2,3),(1,3)}; s=2. Expected: [2,1,3], xác định bằng lần vết kề tăng. Lệnh chạy: python demo.py --example 2. Actual: [2,1,3]. Chèn A02 và giải thích khác biệt về trạng thái/nhánh xử lý so với VD01.

VD02. Input: n=3; E=[[1, 2], [2, 3], [1, 3]]; s=2. Expected: [2, 1, 3].
Lệnh: python demo.py --example 2
Actual: Thứ tự: [2, 1, 3]
Kết quả được kiểm lại tại C trong A10a/A10b.

Cho hình sang trang tiếp theo nếu hai ảnh không đủ rõ; không thu nhỏ chữ đầu ra để giữ một trang.



Hình 4.2 Ví dụ 2 VD02

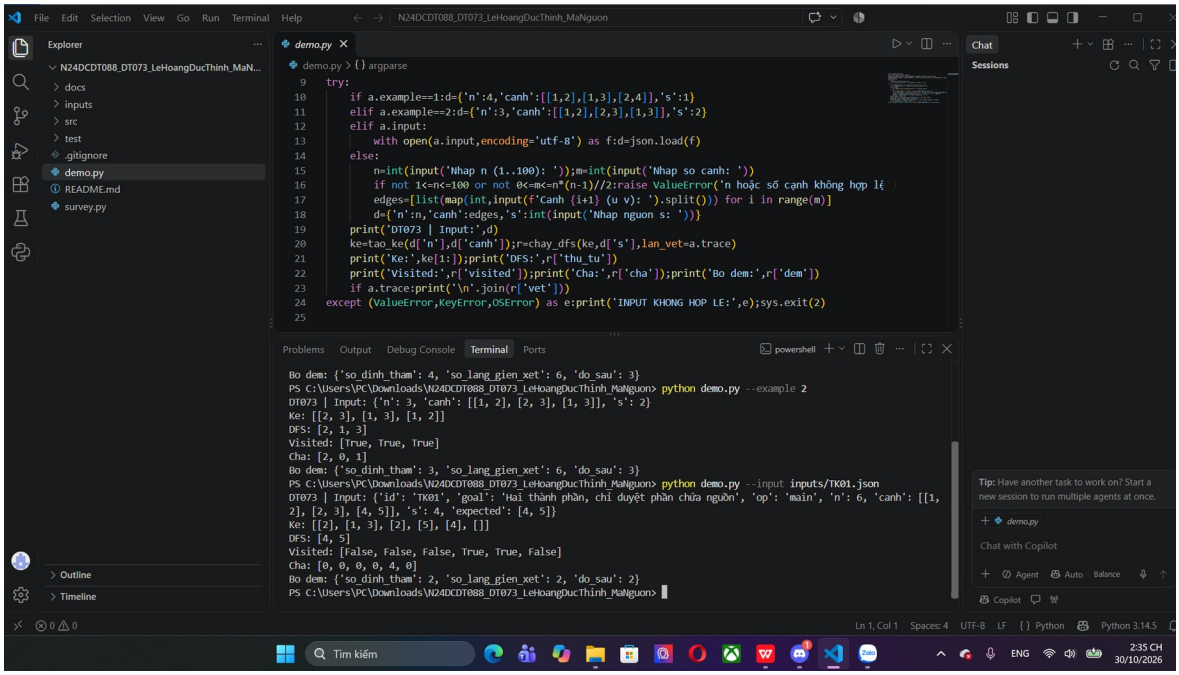
Ảnh: docs/anh_ket_qua/A02.png. Từ nguồn 2 trên tam giác, DFS chọn 1 trước, sau đó tới 3 nên expected là [2,1,3]. Các cạnh quay về đỉnh đã đánh dấu phải được bỏ qua, vì vậy chu trình không làm đỉnh xuất hiện lần hai.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

Nêu mục đích, input đầy đủ và cách tính expected trước khi chạy. Chèn A03, đối chiếu actual và chỉ ra bước quan trọng.

TK01. Input: n=6; E=[[1, 2], [2, 3], [4, 5]]; s=4. Expected: [4, 5].
Lệnh: python demo.py --input inputs/TK01.json
Actual: Thứ tự: [4, 5]
Kết quả được kiểm lại tại C trong A10a/A10b.



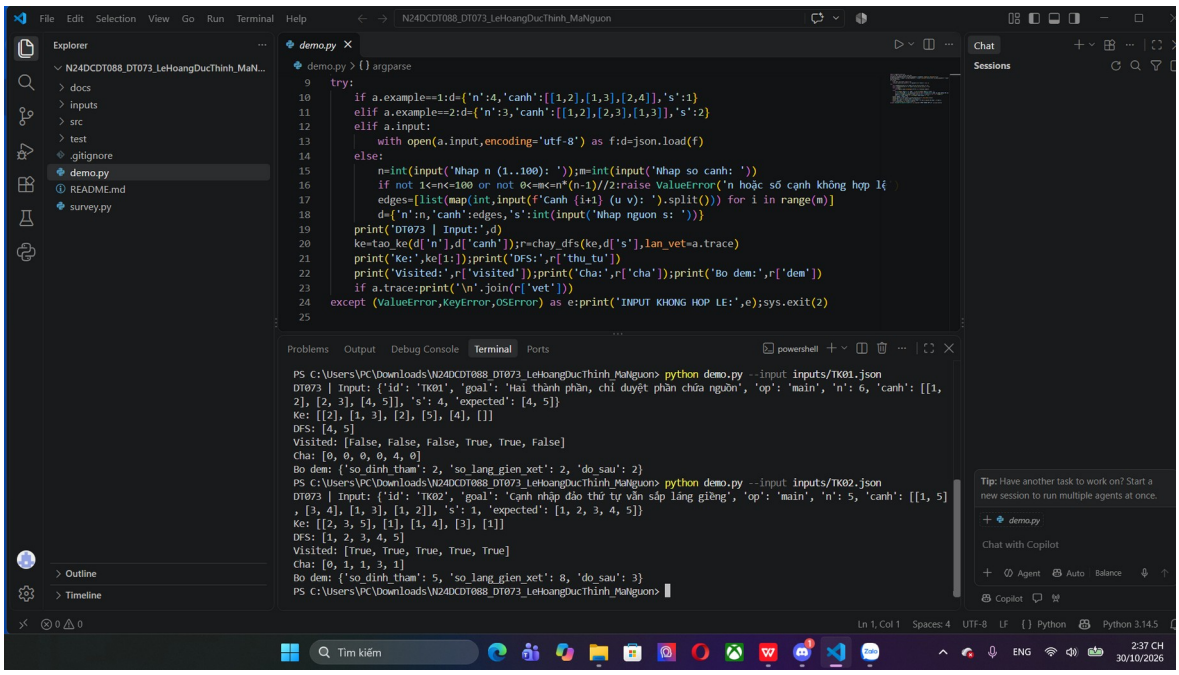
Hình 4.3 Ca tự thiết kế 1 TK01

Ảnh: docs/anh_ket_qua/A03.png. Với nguồn 4, chỉ có 4 và 5 nối với nhau; các đỉnh 1,2,3 và 6 không đạt được từ nguồn. Expected [4,5] và visited=[False,False,False,True,True,False] kiểm việc không tự duyệt thành phần khác.

4.4.2. Ca TK02

Thiết kế input có diễn tiến khác TK01 và hai ví dụ; giải thích vì sao ca bổ sung khả năng phát hiện lỗi. Chèn A04, ghi kết luận dựa expected/actual.

TK02. Input: n=5; E=[[1, 5], [3, 4], [1, 3], [1, 2]]; s=1. Expected: [1, 2, 3, 4, 5].
Lệnh: python demo.py --input inputs/TK02.json
Actual: Thứ tự: [1, 2, 3, 4, 5]
Kết quả được kiểm lại tại C trong A10a/A10b.



Hình 4.4 Ca tự thiết kế 2 TK02

Ảnh: docs/anh_ket_qua/A04.png. Dù thứ tự nhập cạnh không tăng, tao_ke phải sắp láng giềng trước khi DFS. Expected [1,2,3,4,5] kiểm tính xác định của thứ tự duyệt và việc hoàn tất nhánh 3→4 trước khi thăm 5.

4.5. Ba ca biên phù hợp với đề tài

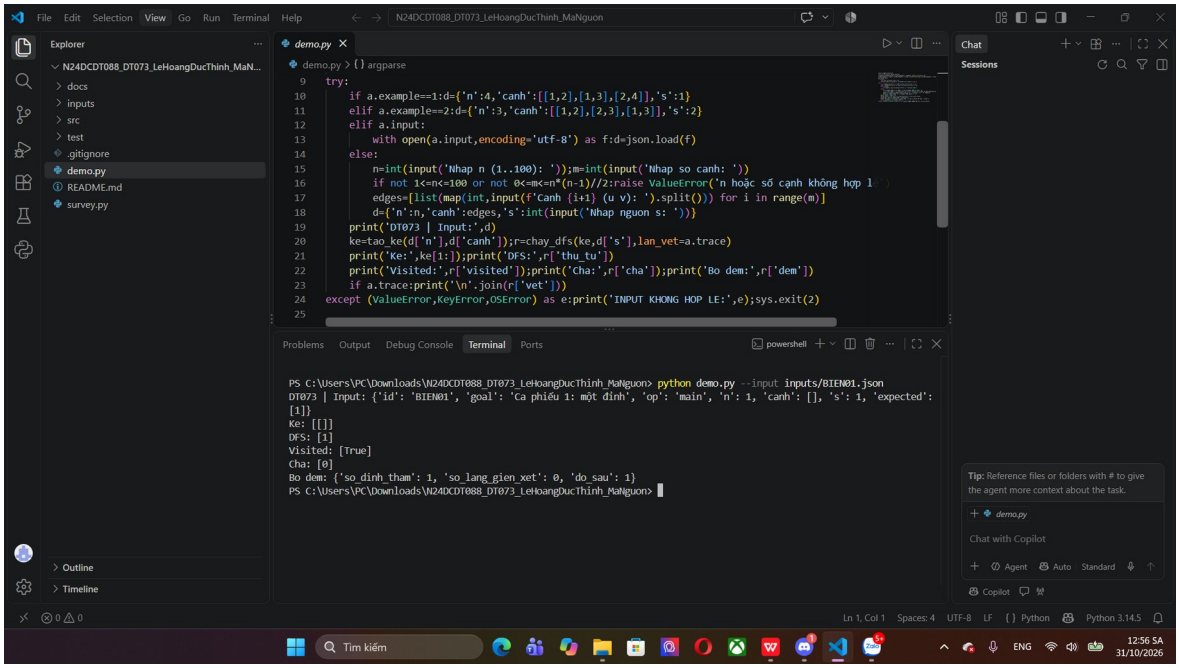
4.5.1. Ca BIEN01

Chọn biên hợp lệ phù hợp với miền phiếu, nêu điều kiện tối thiểu hoặc nhánh đặc biệt được kiểm. Chèn A05, tính expected và giải thích actual.

BIEN01. Input: n=1; E=[]; s=1. Expected: [1].
Lệnh: python demo.py --input inputs/BIEN01.json
Actual: Thứ tự: [1]
Kết quả được kiểm lại tại C trong A10a/A10b.

Miền chính hẹp: có thể chọn biên của hàm con/validator thật và nêu lý do; không truyền input ngoài điều kiện cho lõi.

Một đỉnh không cạnh là input hợp lệ; nguồn vẫn được đánh dấu và xuất một lần.



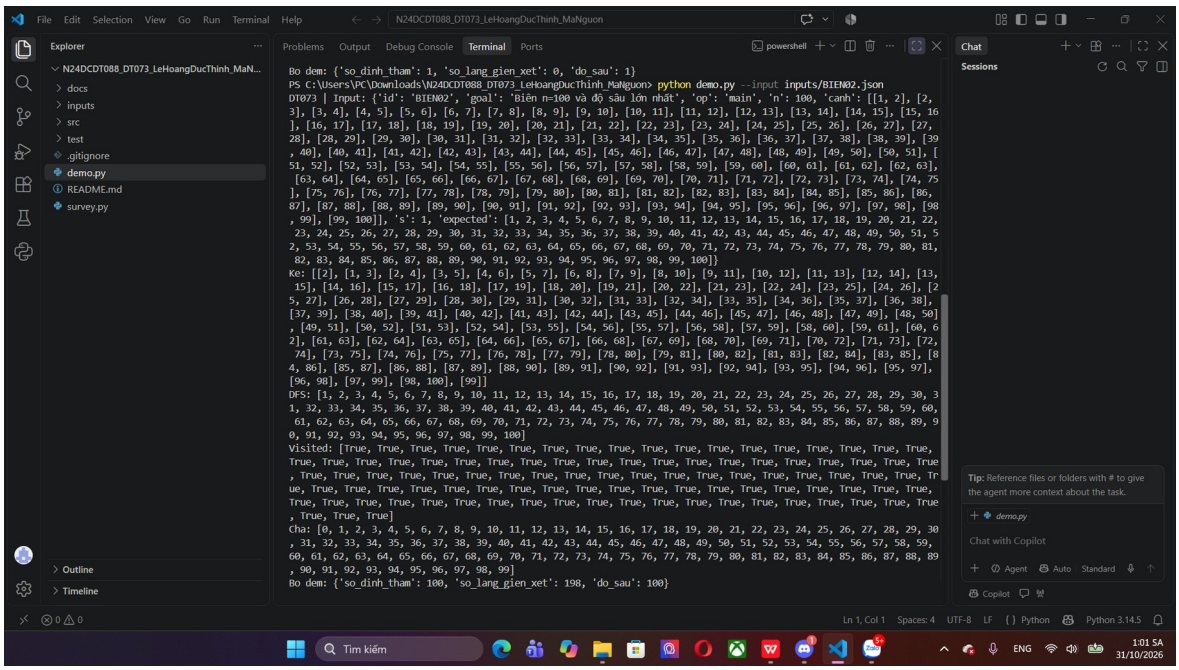
Hình 4.5 Biên 1 BIEN01

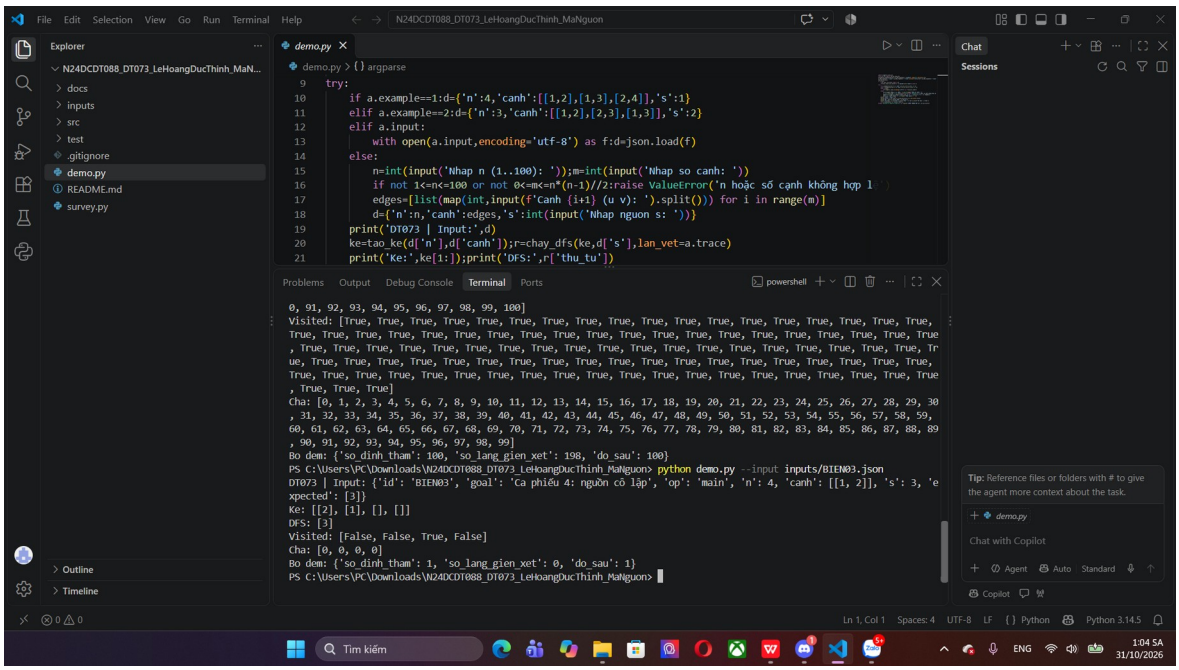
Ảnh: docs/anh_ket_qua/A05.png. Đồ thị một đỉnh không cạnh vẫn có nguồn đạt được chính nó. Expected là [1], visited=[True], cha=[0] và không xét mục kề nào.

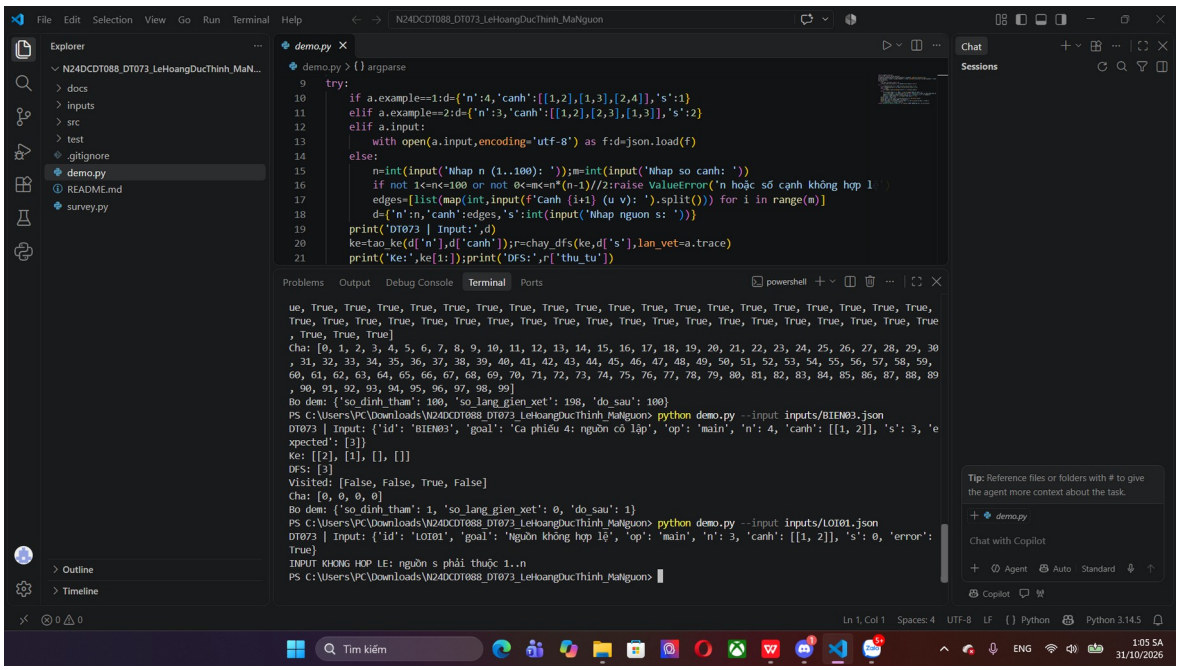
4.5.2. Ca BIEN02

Chọn biên khác BIEN01; nếu dùng một ca bắt buộc trên phiếu thì nêu số thứ tự ca đó. Chèn A06 và giải thích kết quả.

BIEN02. Input: n=100; E={(i,i+1): i=1..99}; s=1. Expected: [1,2,…,100].
Lệnh: python demo.py --input inputs/BIEN02.json
Actual: Thứ tự [1,2,…,100]; 100 đỉnh được thăm.
Kết quả được kiểm lại tại C trong A10a/A10b.


Bo dem: {'so_dinh_tham': 1, 'so_lang_gien_xet': 0, 'do_sau': 1}
PS C:\Users\PC\Downloads\N24DCDT088_DT073_LeHoangDucThinh_MaNguon> python demo.py --input inputs/BIEN02.json
DT073 | Input: {'id': 'BIEN02', 'goal': 'Biên n=100 và độ sâu lớn nhất', 'op': 'main', 'n': 100, 'canh': [[1, 2], [2, 3], [3, 4], [4, 5], [5, 6], [6, 7], [7, 8], [8, 9], [9, 10], [10, 11], [11, 12], [12, 13], [13, 14], [14, 15], [15, 16], [16, 17], [17, 18], [18, 19], [19, 20], [20, 21], [21, 22], [22, 23], [23, 24], [24, 25], [25, 26], [26, 27], [27, 28], [28, 29], [29, 30], [30, 31], [31, 32], [32, 33], [33, 34], [34, 35], [35, 36], [36, 37], [37, 38], [38, 39], [39, 40], [40, 41], [41, 42], [42, 43], [43, 44], [44, 45], [45, 46], [46, 47], [47, 48], [48, 49], [49, 50], [50, 51], [51, 52], [52, 53], [53, 54], [54, 55], [55, 56], [56, 57], [57, 58], [58, 59], [59, 60], [60, 61], [61, 62], [62, 63], [63, 64], [64, 65], [65, 66], [66, 67], [67, 68], [68, 69], [69, 70], [70, 71], [71, 72], [72, 73], [73, 74], [74, 75], [75, 76], [76, 77], [77, 78], [78, 79], [79, 80], [80, 81], [81, 82], [82, 83], [83, 84], [84, 85], [85, 86], [86, 87], [87, 88], [88, 89], [89, 90], [90, 91], [91, 92], [92, 93], [93, 94], [94, 95], [95, 96], [96, 97], [97, 98], [98, 99], [99, 100]], 's': 1, 'expected': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]}
Ke: [[2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 7], [6, 8], [7, 9], [8, 10], [9, 11], [10, 12], [11, 13], [12, 14], [13, 15], [14, 16], [15, 17], [16, 18], [17, 19], [18, 20], [19, 21], [20, 22], [21, 23], [22, 24], [23, 25], [24, 26], [25, 27], [26, 28], [27, 29], [28, 30], [29, 31], [30, 32], [31, 33], [32, 34], [33, 35], [34, 36], [35, 37], [36, 38], [37, 39], [38, 40], [39, 41], [40, 42], [41, 43], [42, 44], [43, 45], [44, 46], [45, 47], [46, 48], [47, 49], [48, 50], [49, 51], [50, 52], [51, 53], [52, 54], [53, 55], [54, 56], [55, 57], [56, 58], [57, 59], [58, 60], [59, 61], [60, 62], [61, 63], [62, 64], [63, 65], [64, 66], [65, 67], [66, 68], [67, 69], [68, 70], [69, 71], [70, 72], [71, 73], [72, 74], [73, 75], [74, 76], [75, 77], [76, 78], [77, 79], [78, 80], [79, 81], [80, 82], [81, 83], [82, 84], [83, 85], [84, 86], [85, 87], [86, 88], [87, 89], [88, 90], [89, 91], [90, 92], [91, 93], [92, 94], [93, 95], [94, 96], [95, 97], [96, 98], [97, 99], [98, 100], [99]]
DFS: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]
Visited: [True, True, True,


demo.py
try:
if a.example==1:d={'n':4,'canh':[[1,2],[1,3],[2,4]],'s':1}
elif a.example==2:d={'n':3,'canh':[[1,2],[2,3],[1,3]],'s':2}
elif a.input:
with open(a.input,encoding='utf-8') as f:d=json.load(f)
else:
n=int(input('Nhap n (1..100): '));m=int(input('Nhap so canh: '))
edges=[list(map(int,input(f'Canh {i+1} (u v): ').split())) for i in range(m)]
d={'n':n,'canh':edges,'s':int(input('Nhap nguon s: '))}
print('DT073 | Input:',d)
ke=tao_ke(d['n'],d['canh']);r=chay_dfs(ke,d['s'],lan_vet=a.trace)
print('Ke:',ke[1:]);print('DFS:',r['thu_tu'])
0, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]
Visited: [True, True, True,



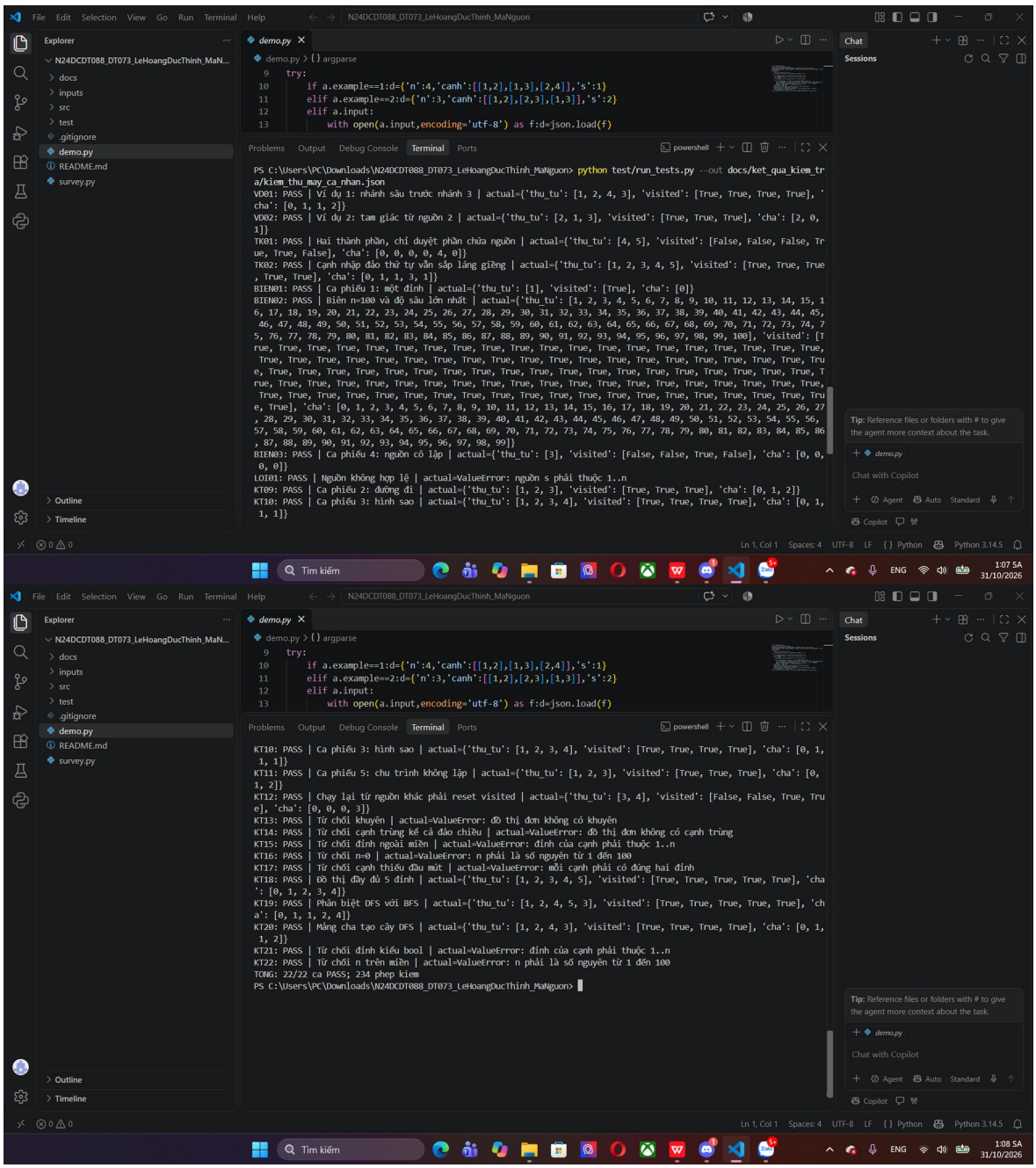
Hình 4.8 Input không hợp lệ LOI01

Ảnh: docs/anh_ket_qua/A08.png. Nguồn s=0 không thuộc tập nhãn 1..3 nên phải bị từ chối trước khi gọi DFS. Ca kiểm đạt PASS khi nhận ValueError đúng dự kiến, không phải khi thuật toán chấp nhận đỉnh 0.

4.7. Kết quả toàn bộ kiểm thử

Chạy lại một lần toàn bộ >=12 ca kiểm thử độc lập; ghi số ca PASS, FAIL và số assert riêng nếu có. Chèn A09; liên hệ với Bảng 4.3, chỉ rõ lỗi còn lại thay vì tự kết luận đạt toàn bộ.

Lệnh: python test/run_tests.py --out docs/ket_qua_kiem_tra/kiem_thu_may_ca_nhan.json
Máy cá nhân: Python 3.14.5; 22/22 ca PASS, 0 FAIL, 234 phép kiểm.
SHA-256 của src/dfs.py trong log khớp tệp mã đính kèm.


File Edit Selection View Go Run Terminal Help
N24DCDT088_DT073_LeHoangDucThinh_MaNguon
Explorer
N24DCDT088_DT073_LeHoangDucThinh_MaN...
docs
inputs
src
test
.gitignore
demo.py
README.md
survey.py
Outline
Timeline
demo.py
demo.py > ... argparse
try:
if a.example==1:d={'n':4,'canh':[[1,2],[1,3],[2,4]],'s':1}
elif a.example==2:d={'n':3,'canh':[[1,2],[2,3],[1,3]],'s':2}
elif a.input:
with open(a.input,encoding='utf-8') as f:d=json.load(f)
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\PC\Downloads\N24DCDT088_DT073_LeHoangDucThinh_MaNguon> python test/run_tests.py --out docs/ket_qua_kiem_tra/kiem_thu_may_ca_nhan.json
VD01: PASS | Ví dụ 1: nhánh sâu trước nhánh 3 | actual={'thu_tu': [1, 2, 4, 3], 'visited': [True, True, True, True], 'cha': [0, 1, 1, 2]}
VD02: PASS | Ví dụ 2: tam giác từ nguồn 2 | actual={'thu_tu': [2, 1, 3], 'visited': [True, True, True], 'cha': [2, 0, 1]}
TK01: PASS | Hai thành phần, chỉ duyệt phần chứa nguồn | actual={'thu_tu': [4, 5], 'visited': [False, False, False, True, True, False], 'cha': [0, 0, 0, 0, 4, 0]}
TK02: PASS | Cạnh nhập đảo thứ tự vẫn sắp láng giềng | actual={'thu_tu': [1, 2, 3, 4, 5], 'visited': [True, True, True, True, True], 'cha': [0, 1, 1, 3, 1]}
BIEN01: PASS | Ca phiếu 1: một đỉnh | actual={'thu_tu': [1], 'visited': [True], 'cha': [0]}
BIEN02: PASS | Biên n=100 và độ sâu lớn nhất | actual={'thu_tu': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100], 'visited': [True, True, True, True,

4.8.2. Kết quả và giới hạn nhận xét

Điền số liệu thật, nhận xét xu hướng so với phân tích; giải thích điểm khác do dữ liệu đầu vào. Không kết luận vượt bằng chứng.

Lệnh khảo sát: python survey.py. Với mỗi n=5,20,50, dựng đường đi có cạnh (i,i+1), 1≤i<n, nguồn s=1 và tắt trace. Mỗi đỉnh được thăm một lần; mỗi cạnh có hai mục kề nên số mục xét lần lượt 8,38,98 và độ sâu lần lượt 5,20,50. Phép đếm chỉ bao gồm sự kiện trong DFS, không gồm tạo cạnh, sort hoặc in. Số liệu này kiểm tính nhất quán của bộ đếm với cấu trúc đường đi, không thay chứng minh độ phức tạp.

Bảng 4.4 Khảo sát Q1 Q2 Q3

Chọn Q1, Q2, Q3 trong đúng miền phiếu. Nếu được tăng quy mô, ghi rõ tham số và ba giá trị; với nhiều bản cài, tôn trọng giới hạn riêng của bản chậm. Nếu kích thước cố định hoặc miền rất hẹp, dùng ba cấu hình/độ khó/giá trị điều khiển hợp lệ và giải thích yếu tố thay đổi. Không tự mở rộng miền chỉ để có ba cỡ; không dùng ba cấu hình cùng kích thước làm chứng minh tốc độ tăng theo kích thước.

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	n=5; đường đi, m=4,s=1	n	m=4; so_dinh_tham=5; so_lang_gien_xet=8; do_sau=5	Tắt trace; xem phân tích 3.6
Q2	n=20; đường đi, m=19,s=1	n	m=19; so_dinh_tham=20; so_lang_gien_xet=38; do_sau=20	Tắt trace; xem phân tích 3.6
Q3	n=50; đường đi, m=49,s=1	n	m=49; so_dinh_tham=50; so_lang_gien_xet=98; do_sau=50	Tắt trace; xem phân tích 3.6

Định nghĩa chính xác phép toán đếm hoặc phần việc đo. Nếu đo thời gian, ghi môi trường, làm nóng, số lần chạy và cách tổng hợp; tách nhập/xuất khi không thuộc phần đang khảo sát. Có thể đếm phép toán thay cho đo thời gian.

Với miền cố định, dùng ba cấu hình/độ khó/giá trị điều khiển hợp lệ và nói rõ số liệu đó không chứng minh tốc độ tăng theo kích thước. Không tự tăng kích thước hoặc miền tham số trái phiếu.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Ghi URL repo, SHA chạy C và permalink hàm chính/test tại C. SHA nộp S ghi trong ThongTinNop.txt nộp rời. Nếu mã/test/input đổi sau C thì chạy lại phần chịu ảnh hưởng.

URL repo:
https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh
SHA C theo ảnh clone A10a: 0347b0a26d4137b2efe6a4a37920262f9d85091b
Permalink thuật toán:
https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh/blob/0347b0a26d4137b2efe6a4a37920262f9d85091b/src/dfs.py
Permalink kiểm thử:
https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh/blob/0347b0a26d4137b2efe6a4a37920262f9d85091b/test/run_tests.py

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

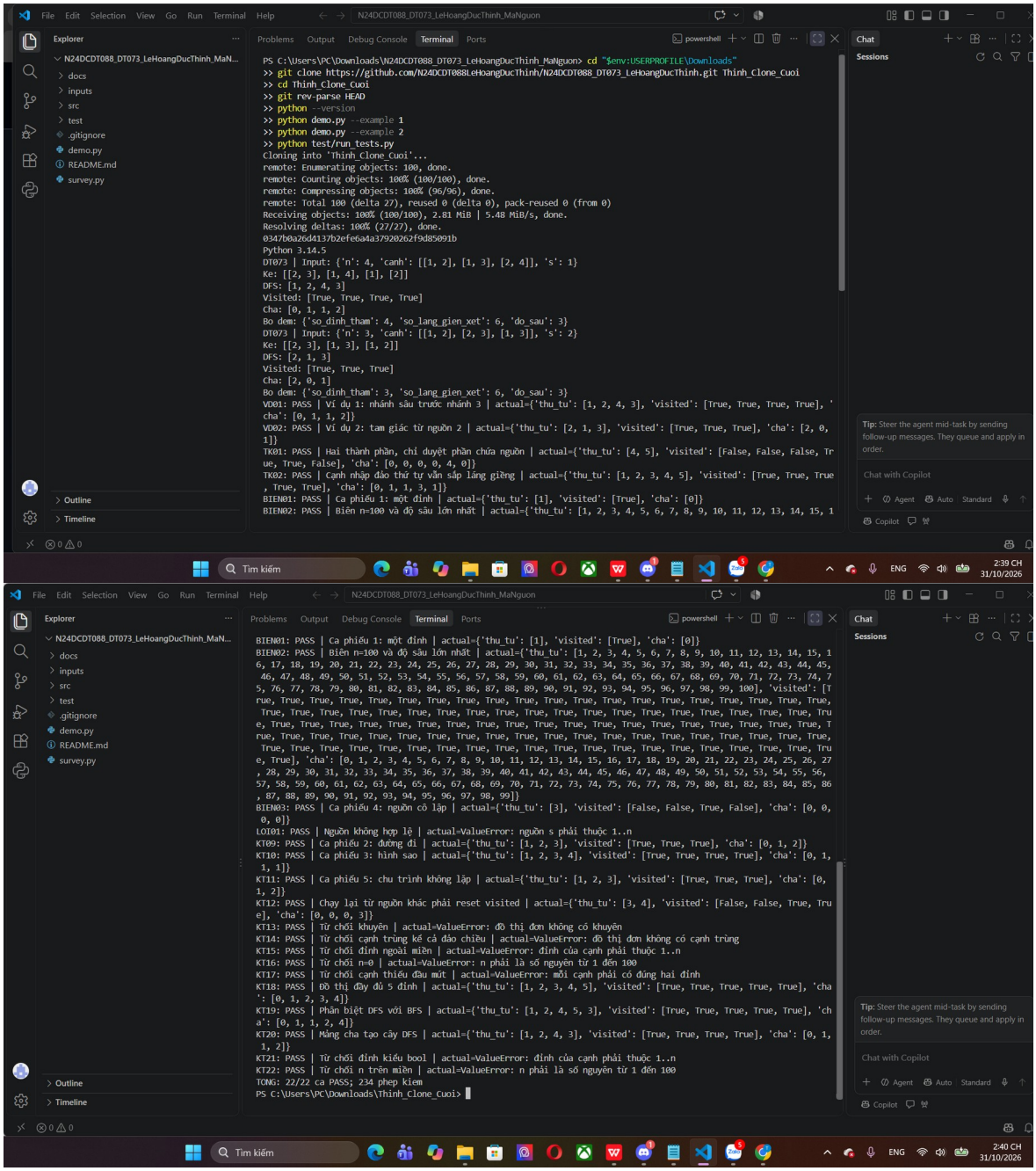
Thông tin	Giá trị thật phải điền
URL repo	https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh
Full SHA chạy ảnh C	0347b0a26d4137b2efe6a4a37920262f9d85091b
SHA S ghi TXT nộp rời	Chưa chốt S; bổ sung vào TXT sau khi cập nhật kho và clone chạy lại.
Permalink hàm chính	https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh/blob/0347b0a26d4137b2efe6a4a37920262f9d85091b/src/dfs.py
Mã/kiểm thử giữa C và S	A10a/A10b xác nhận clone C chạy hai ví dụ và toàn bộ test thành công. Nếu mã, test hoặc input đổi sau C thì chạy lại phần chịu ảnh hưởng trước khi chốt S.
Lệnh chạy lại	python demo.py --example 1 python demo.py --example 2 python test/run_tests.py

4.9.2. Clone sạch và tái hiện kết quả

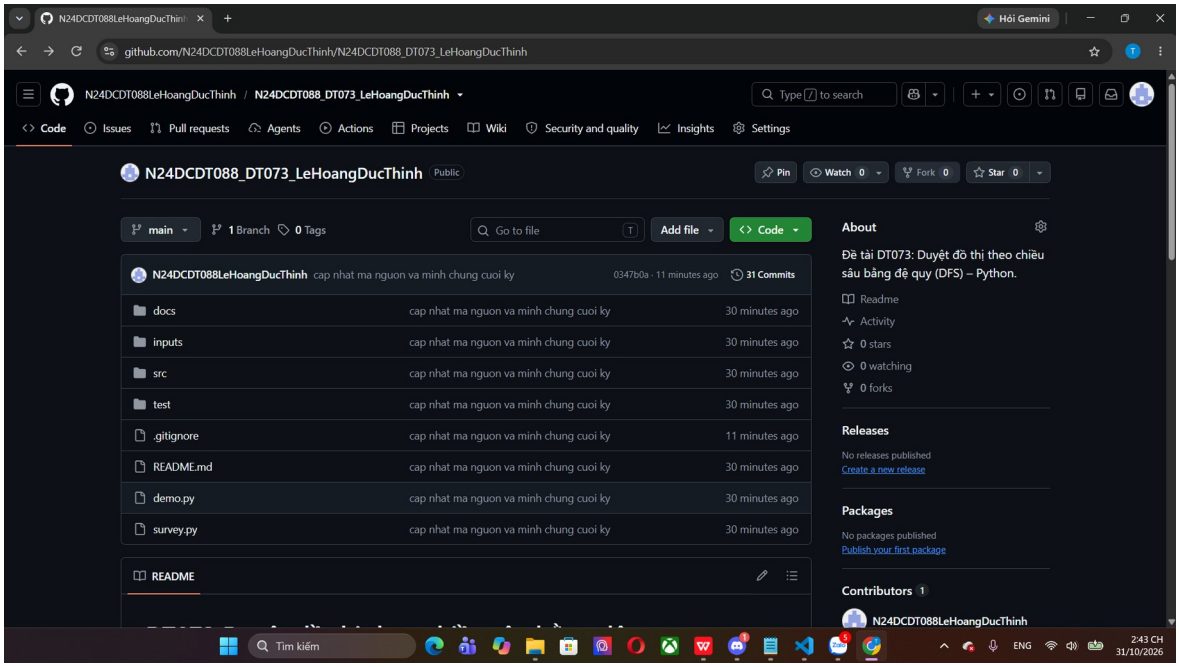
Chụp A10 khi clone C vào thư mục mới, kiểm HEAD, phiên bản và chạy hai ví dụ/toàn bộ test theo README; A11 là GitHub thật tại C. Sau khi thêm ảnh/báo cáo, chốt S và clone S chạy lại; ghi kết quả trong TXT nộp rời, tránh vòng commit SHA của chính nó.

Ảnh A10a/A10b: clone vào Thinh_Clone_Cuoi; HEAD=0347b0a26d4137b2efe6a4a37920262f9d85091b; Python 3.14.5. VD01 có thứ tự [1,2,4,3]; VD02 có thứ tự [2,1,3]. Bộ kiểm thử đạt 22/22 PASS, 0 FAIL, 234 phép kiểm.

Tăng trang nếu bảng hoặc ảnh khó đọc. Giữ SHA C trong báo cáo; S ghi TXT nộp rời. Không thu ảnh quá nhỏ.


PS C:\Users\PC\Downloads\N24DCDT088_DT073_LeHoangDucThinh_MaNguon> cd "$env:USERPROFILE\Downloads"
>> git clone https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh.git Thinh_Clone_Cuoi
>> cd Thinh_Clone_Cuoi
>> git rev-parse HEAD
>> python --version
>> python demo.py --example 1
>> python demo.py --example 2
>> python test/run_tests.py
Cloning into 'Thinh_Clone_Cuoi'...
remote: Enumerating objects: 100, done.
remote: Counting objects: 100% (100/100), done.
remote: Compressing objects: 100% (96/96), done.
remote: Total 100 (delta 27), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (100/100), 2.81 MiB | 5.48 MiB/s, done.
Resolving deltas: 100% (27/27), done.
0347b0a26d4137b2efe6a4a37920262f9d85091b
Python 3.14.5
DT073 | Input: {'n': 4, 'canh': [[1, 2], [1, 3], [2, 4]], 's': 1}
Ke: [[2, 3], [1, 4], [1], [2]]
DFS: [1, 2, 4, 3]
Visited: [True, True, True, True]
Cha: [0, 1, 1, 2]
Bo dem: {'so_dinh_tham': 4, 'so_lang_gien_xet': 6, 'do_sau': 3}
DT073 | Input: {'n': 3, 'canh': [[1, 2], [2, 3], [1, 3]], 's': 2}
Ke: [[2, 3], [1, 3], [1, 2]]
DFS: [2, 1, 3]
Visited: [True, True, True]
Cha: [2, 0, 1]
Bo dem: {'so_dinh_tham': 3, 'so_lang_gien_xet': 6, 'do_sau': 3}
VD01: PASS | Vi dụ 1: nhánh sâu trước nhánh 3 | actual={'thu_tu': [1, 2, 4, 3], 'visited': [True, True, True, True], 'cha': [0, 1, 1, 2]}
VD02: PASS | Vi dụ 2: tam giác từ nguồn 2 | actual={'thu_tu': [2, 1, 3], 'visited': [True, True, True], 'cha': [2, 0, 1]}
TK01: PASS | Hai thành phần, chỉ duyệt phần chứa nguồn | actual={'thu_tu': [4, 5], 'visited': [False, False, False, True, True, False], 'cha': [0, 0, 0, 0, 4, 0]}
TK02: PASS | Cạnh nhập đảo thứ tự vẫn sắp lăng giềng | actual={'thu_tu': [1, 2, 3, 4, 5], 'visited': [True, True, True, True, True], 'cha': [0, 1, 1, 3, 1]}
BIEN01: PASS | Ca phiếu 1: một đỉnh | actual={'thu_tu': [1], 'visited': [True], 'cha': [0]}
BIEN02: PASS | Biên n=100 và độ sâu lớn nhất | actual={'thu_tu': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 1
BIEN01: PASS | Ca phiếu 1: một đỉnh | actual={'thu_tu': [1], 'visited': [True], 'cha': [0]}
BIEN02: PASS | Biên n=100 và độ sâu lớn nhất | actual={'thu_tu': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100], 'visited': [True, True, True, True,

4.9.3. Bằng chứng kho GitHub thật



Hình 4.11 GitHub repo và commit GITHUB

Ảnh A11: kho GitHub công khai, nhánh main, commit 0347b0a; thể hiện docs, inputs, src, test, README.md, demo.py và survey.py. Ảnh gốc: docs/anh_ket_qua/A11.png.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đối chiếu từng mục tiêu Chương 1 với hàm đã cài, kết quả và bằng chứng. Ghi số ca thật, ca lỗi còn lại và khả năng clone chạy lại. Dẫn bảng/hình cụ thể, không chỉ viết đã hoàn thành tốt.

Đã cài đặt thuật toán, lập luận tính đúng, lần vết và khảo sát ba quy mô theo phiếu. Bộ kiểm thử có 22 ca độc lập, đạt 22/22 PASS với 234 phép kiểm. A10a/A10b xác nhận hai ví dụ và toàn bộ test chạy được từ clone sạch tại C; A11 ghi kho GitHub tương ứng. Bước chốt S và kiểm lại S được ghi trong TXT nộp rời.

5.2. Giới hạn và bài học

Nêu miền dữ liệu, nhánh chưa hỗ trợ nếu có, giới hạn khảo sát và vấn đề đã gặp. Giải thích cách phát hiện/sửa một lỗi tiêu biểu bằng bảng lần vết hoặc test.

Bản cài dành cho đồ thị vô hướng đơn, tối đa 100 đỉnh; không giải đồ thị có trọng số hoặc tự duyệt toàn bộ thành phần. Khi mở rộng tới đường đi rất dài, đệ quy có thể chạm giới hạn ngăn xếp. Điểm dễ sai là thời điểm đánh dấu, khởi tạo lại và bỏ qua chi phí sắp kề.

5.3. Hướng phát triển phù hợp

Đề xuất 1–3 bước tiếp theo liên quan trực tiếp bài toán. Ghi rõ phần nào đã có kết quả, phần nào mới đề xuất; không coi phần mở rộng chưa làm là bằng chứng hoàn thành.

Có thể bổ sung giao diện vẽ cây DFS từ mảng cha và chế độ duyệt mọi thành phần. DFS lặp bằng ngăn xếp có thể nghiên cứu khi n lớn, nhưng không thay DFS đệ quy bắt buộc trong đề này.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Đặc tả và thuật toán	Đã cài đặt	src/dfs.py; Chương 3	Đúng miền phiếu
Ca kiểm và lần vết	Có 22 ca và bảng lần vết	test/cases.json; Bảng 3.3,4.2	Log máy cá nhân: 22/22 PASS, 234 phép kiểm; ảnh A01–A09.
Khảo sát	Ba quy mô hợp lệ	survey.py; Bảng 4.4	Chỉ đếm phép toán
Ảnh, Git và bảo vệ	Có đủ A01–A11; clone C chạy thành công	A01–A11; log máy cá nhân; clone tại C	Còn chốt S, clone S chạy lại, xác nhận và ký cam đoan, bảo vệ trực tiếp.

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306; mục 6.3.1 và Bài 6.1; trang in 142–143,145; trang PDF 154–155,157.

[2] Hồ Nhựt Minh, phiếu giao đề tài DT073, học phần INT13185 nhóm 03, bộ tài liệu phát ngày 17/10/2026.

[3] Bộ tài liệu PHAT_CHO_SINH_VIEN_2026_10_17: hướng dẫn sinh viên, rubric chung và mẫu báo cáo cá nhân.

[4] Bản giải tham khảo DT076 và README mã BFS trong gói: tham khảo cách tổ chức tệp, thiết kế kiểm thử và quản lý minh chứng; thuật toán chính của báo cáo theo phiếu riêng.

Mỗi nguồn phải được dùng và dẫn trong nội dung; nguồn mã tham khảo ghi rõ phần tiếp nhận và phần tự làm. Không giữ nguồn không sử dụng.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Chèn cây tệp, yêu cầu môi trường, lệnh chạy ví dụ/nhập dữ liệu/test; dẫn README và permalink. Toàn bộ mã nằm trong repo; chỉ đưa đoạn bổ sung cần thiết vào phụ lục.

src/dfs.py; src/__init__.py; demo.py; survey.py; test/cases.json; test/run_tests.py; inputs/; docs/anh_ket_qua/; docs/ket_qua_kiem_tra/; README.md.
python demo.py --example 1
python demo.py --example 2
python demo.py
python test/run_tests.py

PHỤ LỤC B. Danh mục bằng chứng

Liệt kê A01–A11, mã ca, đường dẫn ảnh/log và commit. Dẫn log đầy đủ cho ca có output dài; ảnh trong thân vẫn phải đọc được.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01	docs/anh_ket_qua/A01.png; kiểm lại tại C: 0347b0a
A02	VD02	docs/anh_ket_qua/A02.png; kiểm lại tại C: 0347b0a
A03	TK01	docs/anh_ket_qua/A03.png; kiểm lại tại C: 0347b0a
A04	TK02	docs/anh_ket_qua/A04.png; kiểm lại tại C: 0347b0a
A05	BIEN01	docs/anh_ket_qua/A05.png; kiểm lại tại C: 0347b0a
A06	BIEN02	docs/anh_ket_qua/A06.png; kiểm lại tại C: 0347b0a
A07	BIEN03	docs/anh_ket_qua/A07.png; kiểm lại tại C: 0347b0a
A08	LOI01	docs/anh_ket_qua/A08.png; kiểm lại tại C: 0347b0a
A09	TEST_ALL	docs/anh_ket_qua/A09a.png; A09b.png; kiểm lại tại C: 0347b0a
A10	CLONE	docs/anh_ket_qua/A10a.png; A10b.png / C: 0347b0a
A11	GITHUB	docs/anh_ket_qua/A11.png / C: 0347b0a

PHỤ LỤC C. Tự kiểm trước khi nộp

Đối chiếu checklist 8 của hướng dẫn; xác nhận thông tin cá nhân/đề, >=12 ca, 11 ảnh, ba quy mô hoặc cấu hình hợp lệ, URL/SHA/permalink, clone sạch và DOCX/PDF cùng phiên bản. Đây là xác nhận sản phẩm đã chuẩn bị, không phải điểm chấm.

□ MSSV và đề tài đúng bảng phân công.

□ Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo.

□ ≥12 ca độc lập; đủ ví dụ và ca phiếu; 11 ảnh thật.

□ Có truy vết, tính đúng, phân tích thời gian/bộ nhớ và ba quy mô hoặc cấu hình hợp lệ.

□ URL, SHA C, permalink và quyền truy cập kiểm được; clone chạy lại.

□ Đã thay toàn bộ hướng dẫn/ngoặc vuông; cập nhật trường Word và xuất PDF.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Tiền tố của bài cá nhân N24DCDT088_DT073_LeHoangDucThinh. Dùng họ tên không dấu, không khoảng trắng và đúng MSSV/mã đề trong phân công. Hạn nộp và lịch bảo vệ theo thông báo trên Classroom.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT088_DT073_LeHoangDucThinh_BaoCao.docx	Bản đã điền; cập nhật mục lục; rubric để GV chấm.
PDF	N24DCDT088_DT073_LeHoangDucThinh_BaoCao.pdf	Xuất từ chính bản Word nộp, kiểm lại từng trang.
Mã nguồn	N24DCDT088_DT073_LeHoangDucThinh_MaNguon.zip	Source, demo/nhập, test, README, ảnh/log và phụ thuộc nếu có.
Thông tin	N24DCDT088_DT073_LeHoangDucThinh_ThongTinNop.txt	URL GitHub thật, SHA C/S, permalink, môi trường/lệnh và kết quả chạy lại.

Không nộp nguyên gói mẫu của giảng viên như ZIP cá nhân. Giải nén ZIP của mình ở thư mục mới phải chạy được theo README. Không gói .git, bộ cài, môi trường ảo hoặc cache vào ZIP bài nộp.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh/báo cáo. Clone S sang thư mục mới rồi chạy lại; nếu mã đổi sau C thì cập nhật test/ảnh chịu ảnh hưởng. SHA S ghi trong TXT nộp rời, không phải commit SHA của chính S vào S.

Các thông tin để chép vào TXT

URL GitHub thật
https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh

SHA C đầy đủ 0347b0a26d4137b2efe6a4a37920262f9d85091b
SHA S đầy đủ: Chưa chốt; ghi trong TXT nộp rời.

Permalink hàm chính và test tại C
https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh/blob/0347b0a26d4137b2efe6a4a37920262f9d85091b/src/dfs.py
https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_DT073_LeHoangDucThinh/blob/0347b0a26d4137b2efe6a4a37920262f9d85091b/test/run_tests.py

Môi trường và phiên bản Windows 11; Python 3.14.5; Git 2.55.0.windows.3; thư viện chuẩn.

Lệnh chạy hai ví dụ và bộ test python demo.py --example 1
python demo.py --example 2
python test/run_tests.py

Kết quả clone S: Chưa chốt S; ghi kết quả sau khi clone S chạy lại. Kết quả tại C: VD01 có thứ tự [1,2,4,3]; VD02 có thứ tự [2,1,3]; 22/22 PASS, 234 phép kiểm.

Vào đúng bài tập trên Classroom, đính kèm đủ bốn tệp, mở kiểm tra lại rồi bấm Nộp bài. Kiểm tra trạng thái đã nộp; ảnh rời đã nằm trong báo cáo và ZIP nên không cần gắn lại 11 tệp ảnh riêng.